



Multi-Behavioral Sequential Recommendation

Shereen Elsayed

elsayed@uni-hildesheim.de

Information Systems and Machine
Learning Lab (ISMLL) & VWFS Data
Analytics Research Center (VWFS
DARC), University of Hildesheim
Hildesheim, Germany

Ahmed Rashed

ahmed.rashed@vwfs.io

Data Analytics & AI Engineering
Volkswagen Financial Services AG
Braunschweig, Germany

Lars Schmidt-Thieme

schmidt-thieme@ismll.uni-
hildesheim.de

Information Systems and Machine
Learning Lab (ISMLL) & VWFS Data
Analytics Research Center (VWFS
DARC), University of Hildesheim
Hildesheim, Germany

ABSTRACT

Sequential recommendation models are crucial for next-item prediction tasks in various online platforms, yet many focus on a single behavior, neglecting valuable implicit interactions. While multi-behavioral models address this using graph-based approaches, they often fail to capture sequential patterns simultaneously. Our proposed Multi-Behavioral Sequential Recommendation framework (MBSRec) captures the multi-behavior dependencies between the heterogeneous historical interactions via multi-head self-attention. Furthermore, we utilize a weighted binary cross-entropy loss for precise behavior control. Experimental results on four datasets demonstrate MBSRec's significant outperformance of state-of-the-art approaches. The implementation code is available here ¹.

CCS CONCEPTS

• Information systems → Recommender systems.

KEYWORDS

Sequential Recommendation, Multi-behavior Recommendation

ACM Reference Format:

Shereen Elsayed, Ahmed Rashed, and Lars Schmidt-Thieme. 2024. Multi-Behavioral Sequential Recommendation. In *18th ACM Conference on Recommender Systems (RecSys '24)*, October 14–18, 2024, Bari, Italy. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3640457.3688166>

1 INTRODUCTION

The importance of recommender systems has grown massively in the last decade, addressing challenges in sequential, multi-behavioral, and attribute-aware recommendations. From early matrix factorization [4] to modern models like SASRec [2] and CARCA [6], efforts focus on personalized next-item recommendations. However, these models often overlook insights from various user behaviors. Recent research [1, 8–10, 12, 13] incorporates auxiliary information for improved recommendations. Early graph-based approaches [1, 10]

¹<https://github.com/Shereen-Elsayed/MBSRec>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RecSys '24, October 14–18, 2024, Bari, Italy

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0505-2/24/10
<https://doi.org/10.1145/3640457.3688166>

suffered from poor performance compared to time-aware sequential models. Recent time-aware models [12, 13] started to include the time factor, considerably affecting the model performance compared to the earlier methods.

In this work, we benefit from the sequential aspect of the data. We propose a sequential method for multi-behavior recommendation, which combines different components and weighting functions to build a simple, flexible, and efficient model that can scale to any arbitrary number of behaviors that might exist in a recommendation setting while having minimal impact on training and inference/response time. Our contributions can be summarized as follows:

- We introduce MBSRec, a simple multi-behavior sequential method, using self-attention layers to capture sequential patterns in multi-behavioral data.
- Utilizing weighted binary cross-entropy loss, our model offers fine control over the contribution weight of each behavior.
- In experiments on four multi-behavioral datasets, MBSRec outperformed state-of-the-art methods by up to 19% and demonstrated a batch runtime over 30 times faster than the closest competitor (MBHT [12]).

2 PROBLEM FORMULATION

In sequential multi-behaviour item recommendation task we define a set of users $\mathcal{U} := \{1, \dots, U\}$ and items $\mathcal{V} := \{1, \dots, I\}$, where each user has a sequence of items $S^u = [(v_1, c_1), (v_2, c_2), \dots, (v_{|S^u|}, c_{|S^u|})]$ which represents the chronologically ordered interaction sequence, where each item interaction v_j is associated with item behavior type c_j such as add-to-cart. The main task is to rank a target list of primary behavior items based on their likelihood of being interacted with by the user u at time $t+1$, while other behaviors act as auxiliary behaviors.

3 METHODOLOGY

3.1 Items and Context Embedding

In sequential recommendation, the goal is to recommend the next item based on user interaction sequences. In multi-behavioral scenarios, diverse user interactions are considered beyond historical sales. Each item in the sequence corresponds to a behavior type (e.g., at time t , user u added item v_x to the cart). Item behavior provides contextual information, indicating user preferences. We encode this information sequence and concatenate it to the item's latent features, obtaining the item embedding $v'_j \in \mathbb{R}^d$. Firstly, the

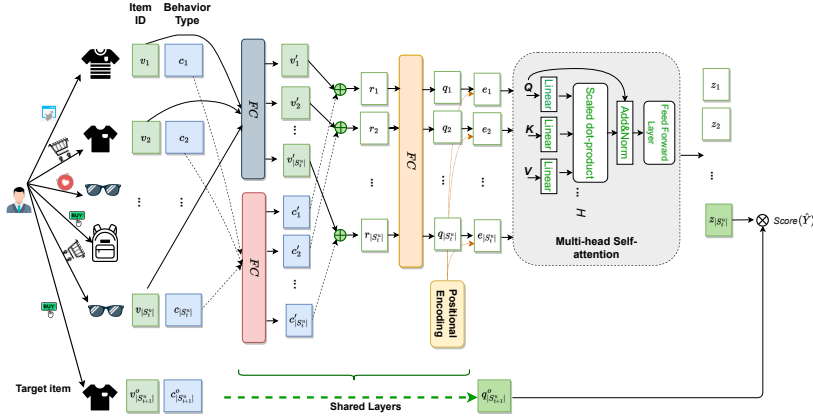


Figure 1: Overall Framework for Multi-Behavioral Sequential Recommendation

item one-hot-encoded vector v_j is fed into a fully connected layer to generate the item embedding. Secondly, the context (behavior types) one-hot-encoded vectors corresponding to the sequence are fed into a fully connected layer to obtain the context embedding as follows:

$$v'_j = v_j W_l + b_l \quad (1) \quad c'_j = c_j W_c + b_c \quad (2)$$

where $W_l \in \mathbb{R}^{I \times d}$ is the weight matrix, I is the number of items, and d is the item embedding dimension. $W_c \in \mathbb{R}^{K \times d}$ is the weight matrix, K is the number of behaviors, and d is the context embedding dimension. The same dimension is used for d , and b_c is the bias term for simplicity. The joint embedding is obtained by column-wise concatenating each item embedding with its context embedding and feeding it to a fully connected layer:

$$r_j = \text{concat}_{col}(v'_j, c'_j) \quad (3) \quad q_j = r_j W_f + b_f \quad (4)$$

where $W_f \in \mathbb{R}^{2d \times d}$ is the weight matrix, where d is the layer embedding dimension, and b_f is the bias term. A learnable positional embedding P_j is added to the item embedding to indicate the item's position within the historical sequence of interactions:

$$e_j = q_j + P_j \quad (5)$$

3.2 Multi-head Self-Attention

The multi-head self-attention block is a key component in the transformer architecture that allows the model to learn dependencies between sequence tokens, words, or items through a scaled-dot product. Here, we utilize multi-head self-attention to encode each item in the user's sequence, creating a rich and expressive representation. The item sequence embeddings $[e_1, e_2, \dots, e_{|S_t^u|}]$ are concatenated row-wise to form the input matrix $E \in \mathbb{R}^{|S_t^u| \times d}$ for the multi-head attention block, as illustrated in Figure 1. Subsequently, we split the input into Query (Q), Key (K), and Value (V) for the multi-head attention layer as follows:

$$\text{Att}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V \quad (6)$$

$$A = \text{SA}(E) = \text{concat}_{col} \left(\text{Att}(EW_h^Q, EW_h^K, EW_h^V) \right)_{h=1:H} \quad (7)$$

where $W_h^Q, W_h^K, W_h^V \in \mathbb{R}^{d \times \frac{d}{H}}$ represent the linear projection matrices of the head at index h , and H is the number of heads. A represents the column-wise concatenation of the attention heads. Finally, we have the point-wise feed-forward layers to obtain the component's final output representations $Z \in \mathbb{R}^{|S_t^u| \times d}$ as follows:

$$Z = \text{FFN}(A) = \text{concat}_{row} \left(\text{ReLU}(A^{(n)} W^{(1)} + b^{(1)}) W^{(2)} + b^{(2)} \right)_{n=1:|S_t^u|} \quad (8)$$

where $W^{(1)}, W^{(2)} \in \mathbb{R}^{d \times d}$ are the weight matrices of the two feed-forward layers, and $b^{(1)}, b^{(2)} \in \mathbb{R}^d$ are their bias vectors. concat_{row} concatenates vectors row-wise, and $Z := [z_1, z_2, \dots, z_{|S_t^u|}]$ is the latent output sequence of items after the multi-head self-attention block.

3.3 Model Prediction

After obtaining the item sequence interaction matrix through the multi-head self-attention block, to score a target item at time $t + 1$, the target item id v^o and behavior type c^o are passed through shared layers for item and context embedding. These embeddings are concatenated and processed through another shared embedding layer to obtain the target item embedding $q_{|S_{t+1}^u|}^o$. The final score is calculated as:

$$\hat{Y} = \sigma(z_{|S_t^u|} \cdot q_{|S_{t+1}^u|}^o) \quad (9)$$

Where σ is a sigmoid function.

3.4 Model Learning

To optimize our multi-behavior recommender model efficiently, we follow the training process of SASRec [2]. The input sequence is the original user interaction sequence excluding the last action $S^u = [(v_1, c_1), (v_2, c_2), \dots, (v_{|S_{t-1}^u|}, c_{|S_{t-1}^u|})]$. This sequence has a fixed maximum length achieved by zero-padding (with item IDs starting from a non-zero value) or truncation. The target sequence is constructed by right-shifting the input sequence and including the last item interaction. For negative sampling, a sequence of random items not in the user's history is generated ($v \notin S^u$).

Dataset	Interactions	User#	Item#	Behavior
Taobao	7,658,926	147,894	99,037	Page View, Favorite, Cart, Buy
Yelp	1,400,000	19,800	22,734	Tip, Dislike, Neutral, Like
MovieLens	9,922,036	677,88	8704	Dislike, Neutral, Like
Tianchi	4,619,389	25,000	500,900	Page View, Favorite, Cart, Buy

Table 1: Datasets Statistics

3.4.1 Weighted binary cross entropy loss. In the multi-behavior paradigm, we address the varying importance of input behaviors by introducing a weighted binary cross-entropy loss. The loss function incorporates the weighting factor α_b to control each behavior's contribution, with K different α_b values for each behavior in the dataset. The weight β adjusts the positive/negative sampling ratio during training. The objective function is formulated as:

$$\mathcal{L} = - \sum_{S^u \in \mathcal{S}} \sum_{t \in [1, 2, \dots, |S^u|]} [\alpha_b \log(\hat{Y}_t^{O^{(+)}}) + \beta (\log(1 - \hat{Y}_t^{O^{(-)}}))] \quad (10)$$

where $\hat{Y}_t^O$ represent output scores for positive and negative samples, respectively. $\mathcal{S}$ is the set of all sequences, α_b denotes behavior weights, and β is the sampling weight in our case we set it to 1.0. Finally, the model is optimized using ADAM optimizer [3].

4 EXPERIMENTS

In this section, we aim to answer the following research questions;

- **RQ1:** How does MBSRec compare to state-of-the-art multi-behavioral and sequential models?
- **RQ2:** How do auxiliary behaviors impact users with limited interactions?
- **RQ3:** What is the effect of varying α_b for different behaviors?

4.1 Datasets

We assess our model on four multi-behavioral datasets. **Taobao**² [9, 13]: a Chinese online shopping platform, includes behaviors like buy, add-to-cart, add-to-favorite, and pageview, with buy considered the target behavior. **Tianchi**³: sourced from Tmall.com, similar to Taobao's behaviors. **Yelp**⁴ [9, 13]: extracted from the Yelp challenge, ratings are categorized as dislike, neutral, and like. **MovieLens**⁵ [9]: centers on movie ratings with behaviors dislike, neutral, and like. Table 1 illustrates the dataset statistics.

4.2 Evaluation Protocol

To ensure a fair comparison, we follow the evaluation protocol of current state-of-the-art methods [9, 13]. Utilizing users' chronologically ordered sequences, we adopt a leave-one-out mechanism for training and validation, excluding the last interaction. For testing, we preserve the last interaction. Each positive item is accompanied by 99 randomly sampled negative items for evaluation. We employ Hit Ratio (HR@N) and Normalized Discounted Cumulative Gain (NDCG@N) as evaluation metrics, with higher values indicating superior model performance. To ensure the result's significance, we report the mean and standard deviation from three runs.

²<https://github.com/akaxlh/MB-GMN/tree/main/Datasets/Tmall>

³<https://tianchi.aliyun.com/competition/entrance/231576/information>

⁴<https://github.com/akaxlh/KHGT/tree/master/Datasets/Yelp>

⁵<https://github.com/akaxlh/KHGT/tree/master/Datasets/MultiInt-ML10M>

4.2.1 Baselines. We compare the proposed method against various recommendation methods to show our model's superiority.

Sequential and Context-Aware Recommendation Methods.

- **SASRec** [2]: A multi-head self-attention model that captures the user's history's sequential pattern.
- **SSE-PT** [7]: A model incorporating user embedding into a personalized transformer with stochastic shared embedding regularization for handling extremely long sequences.
- **CARCA** [6]: A context-aware sequential model that applies cross-attention between user profiles and items to predict the items' scores. For a fair comparison against CARCA, we utilize the same contextual features of MBSRec as input.

Multi-Behavior Recommendation Models.

- **MATN** [8]: A memory-augmented transformer network that utilizes a transformer-based encoder to model behavioral dependencies.
- **MB-GCN** [1]: This model learns node's representation using a graph-convolutional network.
- **MB-GMN** [10]: Learns the multi-behavior dependencies through graph meta-network.
- **KHGT** [9]: This model employs a knowledge graph hierarchical transformer to capture the behavior dependencies.
- **MB-STR** [13]: This model employs a multi-behavior sequential transformer to learn several behaviors.
- **MBHT** [12]: Hypergraph-enhanced transformer model that utilizes self-attention to model behavioral dependencies.
- **KMCLR** [11]: This model incorporates contrastive learning into the multi-behavior recommendation paradigm to address data sparsity issues in input data.

4.3 Model Performance (RQ1)

MBSRec outperforms various state-of-the-art models SASRec and SSE-PT and showcases robustness compared to MATN, MB-GCN, MB-GMN, and KHGT, particularly on the Taobao dataset. It significantly improved HR by 9.5% on Taobao and 2.99% on Tianchi, outshining CARCA across all datasets. In a broader comparison with multi-behavioral models, MBSRec excelled against MB-GCN, MB-GMN, and KHGT on Yelp and MovieLens but addressed the Taobao challenge. Despite recent models like MB-STR and MBHT utilizing multi-head self-attention, MBSRec showcased superior performance with significantly lower runtime. Furthermore, we evaluate the model's performance using the full item set during the test phase. This approach is more representative of real-world scenarios on many platforms, where the goal is to assess the model's ability to rank the test item within the top K list compared to all other items as negative samples, rather than just sampling a small subset of items (e.g., 100) as is commonly done. Table 3 shows our model significance against other baselines such as SASRec [2] and CARCA [6] models on Tianchi dataset.

4.4 Impact of Auxiliary Behaviors and Weighting Factor α_b (RQ2 & RQ3)

We analyze the effect of auxiliary behaviors on user interactions, sorting users by the frequency of their primary interaction. Figure

Method	Taobao		Yelp		MovieLens		Tianchi	
	HR@10	NDCG@10	HR@10	NDCG@10	HR@10	NDCG@10	HR@10	NDCG@10
<i>Sequential and Context-Aware Recommendation Methods</i>								
SASRec [2]	0.390 $\pm$ 4.2E-3	0.249 $\pm$ 8E-4	0.853 $\pm$ 1.7E-3	0.5601 $\pm$ 5.1E-3	0.911 $\pm$ 1E-3	0.668 $\pm$ 5.1E-3	0.659 $\pm$ 3E-3	0.495 $\pm$ 2E-3
SSE-PT [7]	0.393 $\pm$ 2.7E-3	0.232 $\pm$ 3E-4	0.857 $\pm$ 1.5E-2	0.572 $\pm$ 1.8E-2	0.911 $\pm$ 7.1E-3	0.657 $\pm$ 4.5E-3	0.663 $\pm$ 1.2E-2	0.468 $\pm$ 1.3E-2
CARCA [6]	0.769 $\pm$ 4E-3	0.662 $\pm$ 5E-3	0.854 $\pm$ 2E-3	0.589 $\pm$ 4E-3	0.906 $\pm$ 2E-3	0.665 $\pm$ 1E-3	0.713 $\pm$ 4E-4	0.500 $\pm$ 1E-3
<i>Multi-Behavior Recommendation Methods</i>								
MATN [8]	0.354 $\dagger$	0.209 $\dagger$	0.826 $\dagger$	0.530 $\dagger$	0.847 $\dagger$	0.569 $\dagger$	0.714 $\pm$ 7E-4	0.485 $\pm$ 2E-3
MB-GCN [1]	0.369 $\dagger$	0.222 $\dagger$	0.796 $\dagger$	0.502 $\dagger$	0.826 $\dagger$	0.553 $\dagger$	-	-
MB-GMN [10]	0.491 $\dagger$	0.300 $\dagger$	0.861 $\pm$ 8E-3	0.570 $\pm$ 1.2E-2	0.820 $\pm$ 1.1E-3	0.530 $\pm$ 9E-4	0.737 $\pm$ 4.3E-3	0.502 $\pm$ 1.9E-3
KHGT [9]	0.464 $\dagger$	0.278 $\dagger$	0.880 $\dagger$	0.603 $\dagger$	0.861 $\dagger$	0.597 $\dagger$	0.652 $\pm$ 1E-4	0.443 $\pm$ 1E-4
KMCLR [11]	0.455 $\dagger$	0.273 $\dagger$	0.889 $\dagger$	0.603 $\dagger$	-	-	-	-
MB-STR [13]	0.768 $\dagger$	0.608 $\dagger$	0.882 $\dagger$	0.624 $\dagger$	-	-	-	-
MBHT [12]	0.745 $\pm$ 6.1E-3	0.559 $\pm$ 8.4E-3	0.885 $\pm$ 5E-4	0.618 $\pm$ 1.4E-3	0.913 $\pm$ 5.9E-3	0.695 $\pm$ 7E-3	0.725 $\pm$ 6.3E-3	0.554 $\pm$ 4.8E-3
MBSRec (ours)	0.841 $\pm$ 3E-4	0.725 $\pm$ 3E-4	0.898 $\pm$ 2E-3	0.631 $\pm$ 6E-4	0.930 $\pm$ 6E-4	0.713 $\pm$ 1.3E-3	0.755 $\pm$ 9E-4	0.584 $\pm$ 2.7E-3
Improv.(%)	9.50%	19.24%	1.46%	1.12%	1.86%	2.44%	2.99%	5.95%

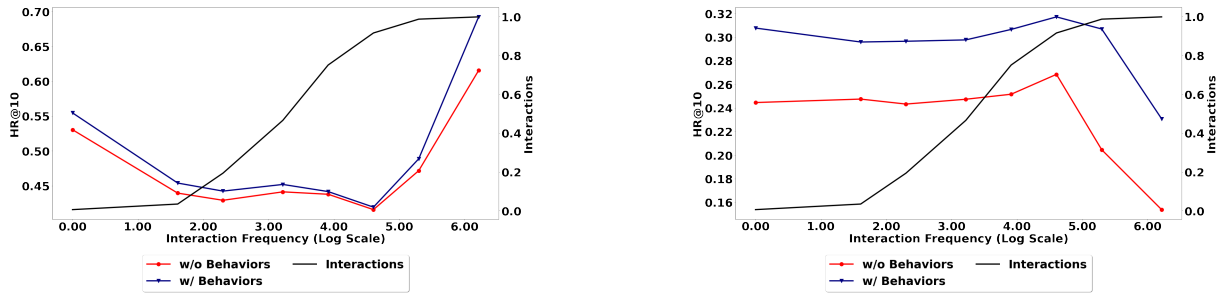
Table 2: Comparative study on Taobao, Yelp, MovieLens, and Tianchi datasets. Published results are indicated by the $\dagger$ symbol.

Figure 2: Effect of adding auxiliary behaviors of different users on Yelp and Tianchi datasets HR@10

Model	HR@10	NDCG@10
SASRec	0.0148	0.0089
CARCA	0.0160	0.0090
MBSRec (ours)	0.0201	0.0096

Table 3: Comparative study on sampling the full item-set in the testing on Tianchi dataset

Model	Average batch runtime (sec)
KHGT	7.830
MBHT	0.957
MB-GMN	0.259
CARCA	0.056
MBSRec (ours)	0.029

Table 4: Runtime analysis on Yelp dataset

2 in the Yelp dataset, focusing on *like* interactions, shows consistent HR improvement for various users. Significant HR enhancement in the Tianchi dataset emphasizes the importance of auxiliary behaviors for diverse user types, especially those with fewer *buy* interactions. In multi-behavioral scenarios, the goal is to assign higher scores to likely user interactions. Introducing α_b for behavior tuning, optimal MovieLens weights are 0.9, 0.1, and 0 for like, neutral, and dislike. Tianchi and Taobao exhibit 0.7, 0.1, 0.1, and 0.1 for buy, cart, favorite, and page views, respectively. Yelp's best weights are 0.3, 0.3, 0.2, and 0.2 for like, tip, neutral, and dislike. This highlights the substantial impact of behavior weights across datasets.

5 RUNTIME AND SCALABILITY ANALYSIS

As shown in Table 4, a runtime analysis on the Yelp dataset with a fixed batch size of 128, our MBSRec model exhibited exceptional efficiency (**0.029** seconds). It outperformed MB-GMN by 88.8%, KHGT by 99.6%, MBHT by 96.9%, and CARCA by 48.2%. These findings show the superior efficiency of our model over current

state-of-the-art models. We add further ablation study about the model scalability to show how the model scales given different input sequence lengths. Table 5 illustrates the model batch processing time and performance given various sequence lengths on the Yelp dataset. The model time scales linearly for the different sequence lengths. However, our model given larger sequence lengths, shows more than 35 times faster than the closest competitor MBHT [12].

6 HYPERPARAMETERS SETTINGS

We ran our experiments using GPU RTX 2070 Super and CPU Xeon Gold 6230 with RAM 256 GB. We used Tensorflow⁶ for implementation; For hyperparameter optimization, we adopted a grid search on the latent embedding size between [30 - 120], on the learning rate between [0.001 and 0.00005], the maximum sequence length between [50 and 200], the batch size we tested [128 and 256], and the dropout rate between [0.2 and 0.6]. The best parameters for dropout are 0.25, a maximum length of 150, an embedding size

⁶<https://www.tensorflow.org>

$ S_t^u $	20	50	70	100	150	200	300	400
Time (sec)	0.015	0.017	0.022	0.024	0.029	0.039	0.064	0.096
HR@10	0.872	0.892	0.896	0.897	0.898	0.900	0.901	0.902
NDCG@10	0.585	0.616	0.627	0.628	0.631	0.630	0.632	0.631

Table 5: Model performance on Yelp dataset given different sequence lengths.

of 85, and a learning rate of 0.0005, respectively, for the Taobao dataset, while for the Tianchi dataset, the dropout is 0.5, the maximum length is 70, embedding size of 50, and learning rate 0.0007. MovieLens dataset dropout is 0.4, the maximum length is 70, the embedding size is 70, and the learning rate is 0.0006. Finally, for the Yelp dataset, the dropout is 0.5, the maximum length is 150, the embedding size is 50, and the learning rate is 0.0003. For all datasets, we employed one attention block with one head with a batch size of 128, as it showed the best performances in our case. Furthermore, we used the ReLU activation function for all the models' layers and the ReLU activation function in the feed-forward layer in the multi-head self-attention block, which has shown better performance. We tuned the α_b between [0 and 1], for Tianchi and Taobao datasets best weight settings are 0.7, 0.1, 0.1, and 0.1 for the buy, cart, favorite, and page views, respectively. While for MovieLens the weights are 0.9, 0.1, and 0 for the like, neutral, and dislike behaviors, and finally for the Yelp dataset 0.3 for (like and tip) and 0.2 for (neutral and dislike) behaviors. Furthermore, we tuned the β factor between [0 and 2], where 1.1 showed the best results. Finally, we employ Adam[3] for optimizing the proposed model.

7 RELATED WORK

In **Sequential Recommendation**, a well-known efficient model SASRec [2], captures the sequential correlation using a transformer encoder. Models such as SSE-PT [14], TiSASRec [5], and S^3 Rec [7] build upon SASRec. CARCA model [6] incorporates contextual information. In **Multi-behavior recommendation**, models like MB-GCN [1] and MB-GMN [10] use convolutional graph networks. Some models employ transformer encoders, like MATN [8], and KHGT [9], which captures user-item interaction characteristics. MBHT [12] utilizes a hyper-graph neural architecture, while MB-STR [13] uses a sequential transformer layer for learning historical behaviors. KMCLR [11] is a recent model, which utilizes a knowledge graph for extracting item representation.

8 CONCLUSION

In this work we present MBSRec, a multi-behavior sequential model for capturing sequential patterns in multi-behavioral data. It encodes input item sequences, leveraging behavior types as contextual features concatenated with item latent features. A multi-head self-attention mechanism captures item dependencies. We enhance the model with a weighted binary cross-entropy loss, giving different weights to behaviors based on significance. Results on four multi-behavioral datasets demonstrate MBSRec's superiority over state-of-the-art methods.

REFERENCES

- [1] Bowen Jin, Chen Gao, Xiangnan He, Depeng Jin, and Yong Li. 2020. Multi-behavior recommendation with graph convolutional networks. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 659–668.
- [2] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [3] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [4] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [5] Jiacheng Li, Yujie Wang, and Julian McAuley. 2020. Time interval aware self-attention for sequential recommendation. In *Proceedings of the 13th international conference on web search and data mining*. 322–330.
- [6] Ahmed Rashed, Shereen Elsayed, and Lars Schmidt-Thieme. 2022. Context and Attribute-Aware Sequential Recommendation via Cross-Attention. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 71–80.
- [7] Liwei Wu, Shuqing Li, Cho-Jui Hsieh, and James Sharpnack. 2020. SSE-PT: Sequential recommendation via personalized transformer. In *Fourteenth ACM Conference on Recommender Systems*. 328–337.
- [8] Lianghao Xia, Chao Huang, Yong Xu, Peng Dai, Bo Zhang, and Liefeng Bo. 2020. Multiplex behavioral relation learning for recommendation via memory augmented transformer network. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2397–2406.
- [9] Lianghao Xia, Chao Huang, Yong Xu, Peng Dai, Xiyue Zhang, Hongsheng Yang, Jian Pei, and Liefeng Bo. 2021. Knowledge-enhanced hierarchical graph transformer network for multi-behavior recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 4486–4493.
- [10] Lianghao Xia, Yong Xu, Chao Huang, Peng Dai, and Liefeng Bo. 2021. Graph meta network for multi-behavior recommendation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 757–766.
- [11] Hongrui Xuan, Yi Liu, Bohan Li, and Hongzhi Yin. 2023. Knowledge Enhancement for Contrastive Multi-Behavior Recommendation. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 195–203.
- [12] Yuhao Yang, Chao Huang, Lianghao Xia, Yuxuan Liang, Yanwei Yu, and Chenliang Li. 2022. Multi-Behavior Hypergraph-Enhanced Transformer for Sequential Recommendation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2263–2274.
- [13] Enming Yuan, Wei Guo, Zhicheng He, Huifeng Guo, Chengkai Liu, and Ruiming Tang. 2022. Multi-Behavior Sequential Transformer Recommender. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1642–1652.
- [14] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 1893–1902.